



UNIVERSIDAD
POLITÉCNICA
DE MADRID



NR sensing testbed & Semantic Compression

8 de mayo de 2025

Pablo Fernández López
Jorge Martín Pérez

1 Semantic Encoding

2 NR sensing testbed

3 Results

1 Semantic Encoding

2 NR sensing testbed

3 Results

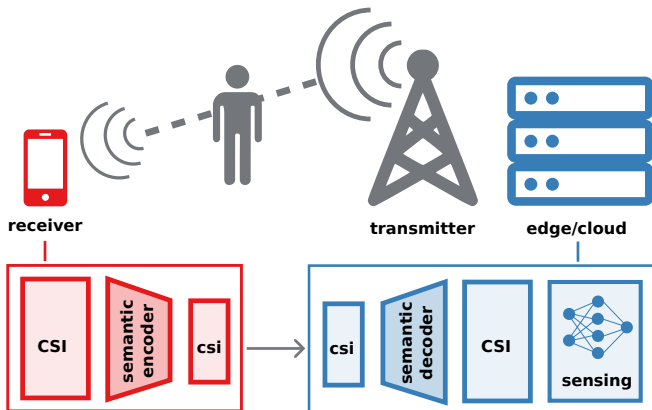


Figure 1: The 5G CSI is compressed through a semantic encoder. The encoded CSI is decoded and processed at the edge/cloud for sensing purposes.

Semantic communications:

transmit necessary information relevant to the specific task

that we want to perform:

Semantic communications:

transmit necessary information relevant to the specific task

that we want to perform: **sensing!**

Semantic communications:

transmit necessary information relevant to the specific task

that we want to perform: **sensing!**

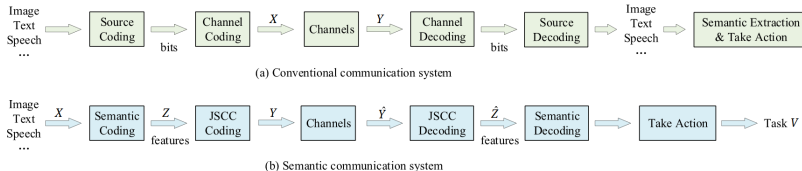


Figure 2: arXiv 2201.01389

What is the SoA doing?

- Semantic Encoders: VAE, codebooks, detection on latent space
- Joint Source Channel Coding as DeepJSCC

What is the SoA doing?

- Semantic Encoders: VAE, codebooks, detection on latent space
- Joint Source Channel Coding as DeepJSCC

The **pitfall**:

- fixed codebooks/compression rates of size K
- different NN architectures per compression size K
- retrain for each compression size K
- sensing detectors retraining for each compression size K

What do we propose?

- semantic encoder with *on the fly* compression rate K
- segment the grid
- maximum compression outside the SSB
- *reuse* pre-trained sensing detectors

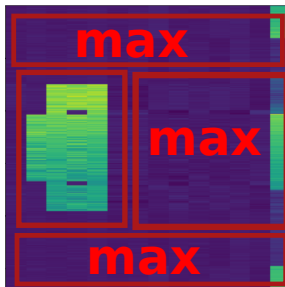


Figure 3

1 Semantic Encoding

2 NR sensing testbed

3 Results

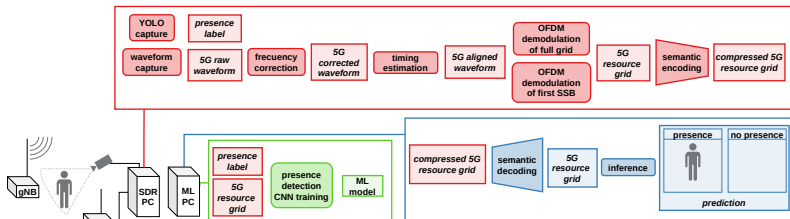
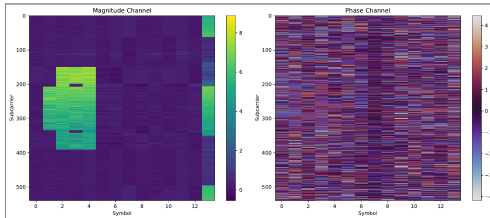


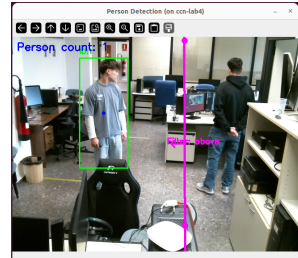
Figure 4: Our testbed allows 5G signal capture, demodulation and labeling. With this data we can train the presence detection ML algorithms and then run inference with semantically compressed resource grids.

Our testbed:

- Allows sub-GHz 5G signal capture using COTS hardware (USRP B210).
- Demodulation process runs in *Python* thanks to the *py3gpp* library.
- Implements GPU acceleration for processing of the captured signals via *CuPy*.
- Performs automatic labeling of the captured frames via *Ultralytics YOLO*.



(a)



(b)

Figure 5: Captured resource grid cropped to SSB region (a) and simultaneous person detection for automatic labeling (b).

With this tools we:

- Adapted a State of the Art Convolutional Neural Network to receive cropped resource grids as input.
- Captured $>19k$ frames for training and ~ 800 for testing after applying semantic encoding.
- Measured detection capabilities degradation and reconstructed files similarity with the original uncompressed files.

1 Semantic Encoding

2 NR sensing testbed

3 Results

Metric for the semantic similarity: Wasserstrein distance.

Experiment: $800 \times 4 \times 16$ (grid, technique, compression)

Metric for the semantic similarity: Wasserstein distance.

Experiment: $800 \times 4 \times 16$ (grid, technique, compression)

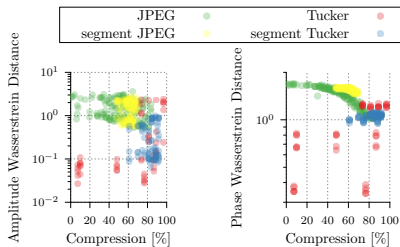


Figure 6

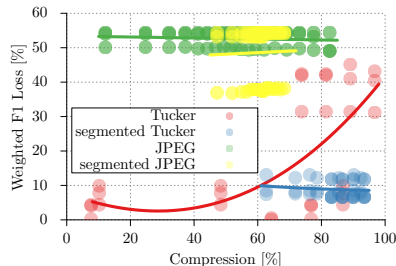


Figure 7

Metric for the semantic similarity: Wasserstein distance.

Experiment: $800 \times 4 \times 16$ (grid, technique, compression)

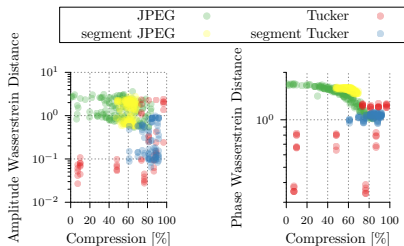


Figure 6

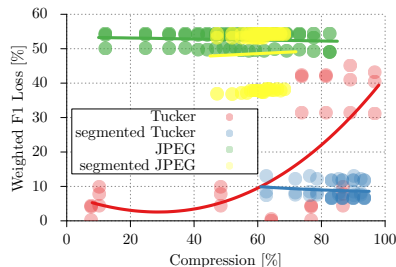


Figure 7

Take-aways:

- more compression doesn't mean more loss
- segmented encoding/compression achieves (i) higher compression; and (ii) better accuracy.